

WEEK 1 · DAY 5 — UI AUTOMATION (PLAYWRIGHT)

Debugging, Stability & Parallel Execution

Develop a debugging mindset and improve script reliability.

Jai Singh

Playwright · TypeScript · Node 24 LTS · Playwright 1.60.x

8-hour session · 3h concepts + 5h hands-on

Where Day 5 sits in the sprint

D1	App Launch & Validation	Setup, execution flow, validation	
D2	Login Workflow	Locators & reliable UI interactions	
D3	Synchronization	Auto-waiting, killing flaky tests	
D4	Navigation, Frames & A11y	Real-world UI flows + Axe checks	
D5	Debugging, Stability & Parallel Execution	Diagnose, stabilise, scale the run	TODAY
D6	Weekly Assessment	Search → Cart → Validate + a11y demo	

Today's flow — and what it builds on

THE DAY (3h concepts → 5h hands-on)

- **The debugging mindset: reproduce before you fix**
- Playwright debug tooling — UI mode, Inspector, page.pause()
- Trace Viewer: the post-mortem for any failure
- Logs & network analysis
- Retry mechanisms — fix vs mask
- Error handling & robust assertions
- Root cause analysis (5 Whys)
- **Parallel execution & test sharding**
- Guided exercises → gated lab → evaluation

BUILDS DIRECTLY ON DAYS 1-4

- D3 synchronization → today we diagnose what still flakes
- D2 locators → trace shows exactly which locator failed
- D4 frames/navigation → harder failures to root-cause
- D1 screenshots/reporting → now traces, videos & retries
- **Sets up D6 assessment — your suite must be green AND fast**

By the end of Day 5 you can...



Reproduce on demand

Turn an intermittent failure into a reliable, repeatable one before touching the code.



Diagnose with artifacts

Read a Playwright trace, video and logs to pinpoint the true root cause.



Stabilise correctly

Use retries and error handling to harden tests — without masking real defects.



Run at scale

Execute the suite in parallel and across shards, then merge reports.

Reproduce before you fix

- **A bug you can't reproduce is a bug you can't prove you fixed.**
- Step 1 — Reproduce: make the failure happen on demand (loop it, slow it, isolate it).
- Step 2 — Observe: capture the evidence (trace, video, logs) at the moment it breaks.
- Step 3 — Hypothesise: form ONE testable theory of the cause.
- Step 4 — Fix the cause, not the symptom.
- Step 5 — Prove it: re-run many times; the failure must be gone, not just rarer.



JAVA / OOP CONNECTION

This is the same loop you used in college with the IntelliJ/Eclipse debugger on a Java NullPointerException: set a breakpoint, inspect state, find the line that's actually null — don't just wrap it in a try/catch and move on. Playwright gives you a debugger for the browser.

ANTI-PATTERN

"It failed once, I re-ran it, it passed — moving on."

That is not a fix. A retry that hides a real race condition will fail in production at 2 a.m. instead of in CI.

Playwright's debugging toolkit



UI Mode

`npx playwright test --ui` — time-travel, watch mode, pick locator. The first tool to reach for.



Inspector

`npx playwright test --debug` — step through actions, headed, with the Playwright Inspector attached.



`page.pause()`

Drop a breakpoint anywhere in the test; execution stops and the Inspector opens at that line.



Headed + `slowMo`

Watch the browser drive in real time; slow each action to see exactly where it diverges.

The post-mortem for any failure

- A trace is a recorded flight-data box of the run.
- DOM snapshots before / action / after each step
- Full network log, console output and source
- Action timeline you can scrub through
- **Capture it cheaply — only when it matters:**
- on-first-retry keeps cost low in CI
- Open with: `npx playwright show-trace trace.zip`

CONFIGURE TRACE CAPTURE

```
// playwright.config.ts
export default defineConfig({
  use: {
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
});
```

 **IN THE IDE** We trigger a failure, then open its trace and scrub to the exact broken step.

Reading a trace beats reading a stack trace: you see what the page actually looked like when the assertion fired.

Screenshot vs video vs trace — and handle with care



Screenshot

Final visual evidence — the page at the moment of failure. Cheapest to capture and read.



Video

Watch movement and timing across the run — useful when the failure is about sequence or animation.



Trace

The full post-mortem: DOM snapshots, actions, network, console and source. Reach for this first.

⚠️ ARTIFACT HYGIENE — THIS MATTERS IN AN ENTERPRISE

- Traces, videos, reports, console logs and screenshots can capture credentials, tokens, URLs and customer-like test data.
- Never upload them to a public location. Share only through approved internal channels, and prefer synthetic / masked data in tests.
- **Treat an artifact like a production log — because that is effectively what it is.**

Logs & network analysis

LISTEN TO THE BROWSER

```
// surface what the page is doing
page.on('console', m =>
  console.log('PAGE LOG:', m.text()));
page.on('pageerror', e =>
  console.log('PAGE ERROR:', e.message));
page.on('requestfailed', r =>
  console.log('FAILED:', r.url(),
    r.failure()?.errorText));

// verbose Playwright API logs
// DEBUG=pw:api npx playwright test
```

- **Most UI 'flakiness' is really a backend 500 or a slow XHR.**
- console — app-level errors the page logged
- pageerror — uncaught JS exceptions
- requestfailed — the network call that never came back
- DEBUG=pw:api — Playwright's own step-by-step log



JAVA / OOP CONNECTION

Same idea as Log4j / SLF4J in your Java framework — structured, levelled logs that turn a silent failure into a readable story. Here the events are first-class objects on the page.

Retry to absorb noise — never to hide bugs

- Retries re-run a failed test up to N times.
- Legitimate use: genuine external noise (a flaky third-party, transient network).
- Abuse: a real race condition that you choose not to fix.
- **Rule: every retry that 'saves' a test is a signal to investigate, not celebrate.**
- Set retries in CI, keep them at 0 locally so you feel the pain and fix it.
- Pair with trace: 'on-first-retry' so the first failure is captured.

RETRIES, SCOPED TO CI

```
// playwright.config.ts
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  reporter: 'html',
  use: { trace: 'on-first-retry' },
});
```

⚠️ FLAKE BUDGET

Track which tests are 'saved' by a retry. A test that only passes on retry is on probation — it gets a root-cause investigation this sprint, not a permanent retry.

Error handling & robust assertions

EXPRESSIVE FAILURES

```
// web-first assertions auto-retry
await expect(page.getByRole('button',
  { name: 'Checkout' })).toBeEnabled();

// soft: collect failures, keep going
await expect.soft(total).toHaveText('$48');
await expect.soft(items).toHaveLength(3);

// custom message for fast triage
expect(orderId, 'order id missing')
  .toBeTruthy();
```

- **Prefer web-first assertions — they auto-wait and retry.**
- `expect.soft()` reports every mismatch in one run, not just the first.
- Custom messages make a CI failure self-explanatory.
- Avoid manual try/catch around actions unless you truly expect failure.



JAVA / OOP CONNECTION

`expect.soft()` is JUnit 5's `assertAll()` — gather all assertion errors and report them together. A custom message is the second arg to `assertEquals(expected, actual, "message")`. Same intent, browser-aware engine.

Root cause analysis — the 5 Whys

- 0 Symptom: 'Checkout' fails ~1 in 10 — opens a stale total.
- 1 Why? We acted before the /cart/total data was applied.
- 2 Why? The button is enabled before that data is ready.
- 3 Why? 'enabled' is not the real readiness signal here.
- 4 Why? A weak text locator + no wait on the response.
- ✓ Root cause → role locator + wait for the real signal.

- **Classify before you fix:**
- Sync issue → wait on the right state
- Locator issue → use role/text, not brittle CSS
- Data issue → isolate / seed deterministic data
- Environment → backend slow or down (a real defect!)
- Test-logic issue → wrong assertion or order
- **The category tells you the fix. Guessing wastes the sprint.**

Parallel execution & test sharding

- **Default:** Playwright runs test FILES in parallel (workers).
- `fullyParallel: true` also parallelises tests WITHIN a file.
- Turn it on only after each test owns its data + setup.
- Sharding splits the whole suite across machines/jobs.
- `--shard=1/3` runs the first third; a CI matrix runs all three.

PARALLEL + SHARDED

```
// config
fullyParallel: true,
workers: process.env.CI ? 4 : undefined,

# CI matrix: shard 1/3, 2/3, 3/3
npx playwright test --shard=1/3
# collect every shard's blob, then merge
npx playwright merge-reports \
  --reporter=html ./all-blob-reports
```

PRE-CONDITION: TEST INDEPENDENCE

Parallelism only works if tests don't share state. Each test sets up its own data and never depends on another test's side-effects — the same encapsulation discipline you apply to a well-designed Java class.

Selenium + Java → Playwright: debug view

Debugging task	Selenium + Java (college)	Playwright + TypeScript
Step through a test	IDE breakpoints + JDB / IntelliJ debugger	page.pause() + Inspector, or --ui mode
Post-mortem of a failure	Manual screenshots; ITestListener on failure	Trace Viewer: DOM, network, console, source
Absorb transient noise	TestNG IRetryAnalyzer	retries in config (CI-scoped)
Run in parallel	TestNG parallel + thread-count; Surefire forks	workers + fullyParallel
Split across machines	Selenium Grid nodes	--shard=i/n + merge-reports
Logging	Log4j / SLF4J appenders	page.on('console' 'pageerror'); DEBUG=pw:api

You already own the concepts. Today swaps the Java debugging muscle for Playwright's built-in, browser-aware tooling.

HANDS-ON · PART 1

Reproduce, then read the evidence

We start from a script that fails intermittently. No guessing — we make it fail reliably, capture a trace, and let the artifacts point to the cause.

SCENARIO

The script that fails 'sometimes'

SYMPTOM

- A 'Add to Cart → Checkout' test passes locally, fails ~1 in 8 CI runs.
- Error: locator click did not register / element not enabled.
- Re-running 'fixes' it — the classic trap.
- **Our job: stop re-running. Reproduce on demand.**

First move: reproduce it. Run the single test in a tight loop until it fails, so we can study a real failure.

BEFORE – FRAGILE

```
// the suspect test (before)
test('checkout', async ({ page }) => {
  await page.goto('/cart');
  // weak text locator; clicks before
  // the cart total has been applied
  await page.getByText('Checkout')
    .click();
  await expect(page)
    .toHaveURL(/\/order\/);
});
```

Then capture a trace on that failure and open it side-by-side with the code.

Reproduce on demand + capture the trace

▶ **IN THE IDE** Trainer drives this live; slides show only the key commands. Follow along on your machine.

TERMINAL

```
# 1) reproduce: loop one test, headed, slow
npx playwright test checkout \
  --project=chromium --repeat-each=20 \
  --headed --workers=1 --trace on

# 2) or step through interactively
npx playwright test checkout --debug

# 3) find the trace, then open it
find test-results -name trace.zip
npx playwright show-trace <path>
```

- **--repeat-each=20 forces the intermittent failure to surface.**
- **--workers=1** removes parallelism while we isolate the cause.
- **--trace on** records every run so we never lose the failing one.
- **In the trace we scrub to the click step: the /cart/total response is still pending, so the total isn't applied yet.**
- That single observation replaces an hour of guessing.

Let the trace name the root cause

1

Open the failing trace

Pick the run that failed in the repeat loop.

2

Scrub to the click action

See the click in full DOM + network context.

3

Inspect the DOM snapshot

Total not yet applied; weak text match.

4

Check the network tab

A /cart/total XHR is still pending.

5

Name the cause

Acted before the real data signal landed.



JAVA / OOP CONNECTION

Reading a trace is reading a stack trace with the heap attached — you don't just see WHERE it threw, you see the full object/DOM state at that instant. It's the browser equivalent of inspecting variables at a breakpoint.

✓ ROOT CAUSE

click() already auto-waits for actionability — but the button is enabled before the cart-total response lands. Fix: a role-based locator + wait for the real signal, then click. Not a retry.

One config: stability + scale

STABILITY + SCALE IN ONE PLACE

```
// playwright.config.ts
import { defineConfig, devices } from
  '@playwright/test';
export default defineConfig({
  fullyParallel: true,
  workers: process.env.CI ? 4 : undefined,
  retries: process.env.CI ? 2 : 0,
  reporter: process.env.CI
    ? [['list'], ['blob']]
    : [['html']],
  use: {
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [ { name: 'chromium',
    use: devices['Desktop Chrome'] } ],
});
```

- **fullyParallel** — max throughput
- workers + retries — only in CI
- blob — mergeable shard output; list adds CI visibility (add junit if needed)
- trace on-first-retry — evidence without slowing every run

Local stays simple (0 retries, 1 reporter) so you feel and fix flakiness; CI scales up and captures evidence.

GUIDED PRACTICE · PART 2

Three exercises, side by side

Try each one first (amber). We reconvene on the solution (green). Type it — don't just read it.

EXERCISE 1

Capture & open a trace on failure

 YOU TRY

~8 min

- **Goal: make a deliberately wrong test fail, then read its trace.**
- 1. Write a test that asserts the cart shows 5 items (it has 1).
- 2. Configure trace: 'retain-on-failure' for this run.
- 3. Run it; let it fail.
- 4. Open the trace and find the exact step + DOM snapshot.
- **Deliverable: the trace screen open at the failing assertion.**

HINTS

- Run a single test by title: `npx playwright test -g "cart shows 5 items" --trace retain-on-failure`
- The trace.zip lands under test-results/
- `npx playwright show-trace <path>`
- Use `--headed` if you also want to watch it live

EXERCISE 1

Solution — trace on failure

✓ SOLUTION

SOLUTION

```
test('cart shows 5 items', async ({ page })
  => {
    await page.goto('/cart');
    // intentionally wrong: cart has 1
    await expect(page.getByTestId('item'))
      .toHaveCount(5); // fails
  });
```

```
# run + capture, then open
npx playwright test \
  tests/cart.spec.ts \
  --trace retain-on-failure
npx playwright show-trace \
  test-results/cart-*/trace.zip
```

- toHaveCount auto-retried for the timeout, then failed — proving it's not a timing issue but a real mismatch.
- The trace's DOM snapshot shows exactly one item rendered.
- **You diagnosed without adding a single console.log.**
- This is the loop you'll use on every real failure today.

EXERCISE 2

Fix the race — cause, not symptom

 YOU TRY

~12 min

- **Goal: stabilise the flaky checkout test WITHOUT a retry.**
- 1. Reproduce with `--repeat-each=20 --workers=1`.
- 2. From the trace, confirm we act before `/cart/total` is applied.
- 3. Use a role locator + wait for the real signal, then click.
- 4. Re-run `--repeat-each=20` — it must pass every time.
- **Deliverable: 20/20 green with retries still at 0 locally.**

HINTS

- `page.getByRole('button', { name: 'Checkout' })`
- Wait for the real signal: `page.waitForResponse(/cart/total)`
- Then click — it already auto-waits for actionability
- 20/20 with 0 retries = cause fixed

EXERCISE 2

Solution — wait for the real signal

✓ SOLUTION

AFTER — STABLE

```
test('checkout', async ({ page }) => {  
  // listen FIRST, then act (Day 3 rule)  
  const total = page.waitForResponse(r =>  
    r.url().includes('/cart/total') &&  
    r.status() === 200);  
  await page.goto('/cart');  
  await total;  
  const btn = page.getByRole('button',  
    { name: 'Checkout' });  
  await expect(btn).toBeEnabled();  
  await btn.click();  
});
```

- We wait for the actual /cart/total response — the real readiness signal, not just 'enabled'.
- The role locator drops the weak text match; click then auto-waits for actionability.
- **No sleeps, no retries, no masking — the race is gone.**
- 20/20 green with retries: 0 proves it.



JAVA / OOP CONNECTION

You replaced a guess with a precondition — like guarding a method on an explicit state check instead of catching the exception it would otherwise throw. Fix the contract, not the crash.

EXERCISE 3

Run the suite in parallel + shards

 YOU TRY

~12 min

- **Goal: prove the suite is independent, then split it.**
- 1. Set `fullyParallel: true` and run all tests.
- 2. If anything breaks, fix the shared-state dependency.
- 3. Run `--shard=1/2`, then `--shard=2/2` (two terminals).
- 4. Collect both shards' blob zips, then merge into one report.
- **Deliverable: one merged report covering both shards, all green.**

HINTS

- `reporter: [['blob']]` so shards can be combined
- `npx playwright test --shard=1/2`
- A failure under parallel = a hidden shared-state bug
- Collect zips into `./all-blob-reports`, then `merge-reports`

EXERCISE 3

Solution — parallel, sharded, merged

✓ SOLUTION

SOLUTION

```
// config
fullyParallel: true,
reporter: [['blob']],

# terminal 1 / CI job 1
npx playwright test --shard=1/2
# terminal 2 / CI job 2
npx playwright test --shard=2/2

# collect every shard's blob zip, then merge
mkdir -p all-blob-reports
cp shard-*/blob-report/*.zip all-blob-reports/
npx playwright merge-reports \
  --reporter=html ./all-blob-reports
```

- If shard runs stay green, your tests are truly independent — the real win.
- Each shard emits its own blob zip; gather them into one folder, then merge into a single HTML report.
- In CI this becomes a 2- or 3-job matrix that cuts wall-clock time roughly in half / in third.

Main lab — stabilise & scale a real suite

THE TASK

- You're handed a 6-test retail suite with 2 flaky tests and no parallelism.
- Reproduce both failures on demand and capture traces.
- Root-cause each one and fix the cause (no blind retries).
- Add error handling + soft assertions where it improves triage.
- Turn on parallel execution and prove independence.
- Shard across 2 jobs and merge into one report.
- **Work through 5 gated milestones — each Gate must pass before you move on.**

RULES OF ENGAGEMENT

- Reproduce before you fix — always.
- retries: 0 locally; you must earn green honestly.
- Every fix is justified from a trace, not a hunch.
- **A test that needs a retry to pass is NOT done.**
- Independence is non-negotiable for parallel runs.

Reproduce both failures + capture traces

- **Objective:** stop the 'sometimes' — make each flaky test fail on demand.
- Isolate each suspect test and loop it serially.
- Capture a trace for every run so the failing one is saved.
- Open both traces; note the failing step for each.

TERMINAL

```
# loop each suspect, serial, with traces
npx playwright test flakyA \
  --repeat-each=20 --workers=1 \
  --trace on
npx playwright test flakyB \
  --repeat-each=20 --workers=1 --trace on

npx playwright show-trace <trace.zip>
```

✓ **GATE** Both tests reproduced failing at least once, with a saved trace.zip opened to the failing step.

Root-cause + fix flaky test #1

- **Objective: fix the timing race — no retries.**
- From the trace, confirm we acted before the data was ready.
- Use a role locator + wait for the real readiness signal.
- Re-run --repeat-each=20; must be 20/20 green.

FIX #1

```
// before – weak locator, acts too early
await page.getByText('Pay').click();
// after – listen first, then trigger
const total = page.waitForResponse(r =>
  r.url().includes('/cart/total') &&
  r.status() === 200);
await page.reload(); // triggers it
await total;
const pay = page.getByRole('button',
  { name: 'Pay' });
await expect(pay).toBeEnabled();
await pay.click();
```

✓ **GATE** Test #1 passes 20/20 with retries: 0 — root cause fixed, justified by the trace.

Root-cause + fix flaky test #2

- **Objective:** fix the shared-state / locator defect.
- Trace shows test #2 depends on data another test created.
- Give it its own setup; switch to a role/text locator.
- Add a soft assertion so all mismatches surface at once.

FIX #2

```
test.beforeEach(async ({ page }) => {  
  await seedCartFor(page); // own data  
});  
const row = page.getByRole('row',  
  { name: /Order #\d+/ });  
await expect.soft(row).toBeVisible();  
await expect.soft(total).toHaveText('$48');
```

✓ **GATE** Test #2 passes 20/20 independently; no reliance on another test's side-effects.

Enable parallel + prove independence

- **Objective:** run the whole suite in parallel, all green.
- Set `fullyParallel: true` and `workers > 1`.
- Any new failure is a hidden shared-state bug — fix it.
- Confirm the suite is order-independent.

PARALLEL

```
// playwright.config.ts
fullyParallel: true,
workers: 4,

npx playwright test
# expect: all tests green, faster wall-clock
```

✓ **GATE** Full suite passes with `fullyParallel + workers: 4` — independence proven.

Shard across 2 jobs + merge report

- **Objective:** split the suite and reassemble one report.
- Switch reporter to blob for shardable output.
- Run shard 1/2 and 2/2 (two terminals or CI jobs).
- Collect both blob zips into one dir, then merge.

SHARD + MERGE

```
reporter: [['blob']],
```

```
npx playwright test --shard=1/2
```

```
npx playwright test --shard=2/2
```

```
# gather zips, then merge the folder
```

```
npx playwright merge-reports \  
  --reporter=html ./all-blob-reports
```

✓ **GATE** One merged HTML report covers both shards, all green — suite is stable AND scalable.

WATCH OUT

Common pitfalls (and the fix)

Pitfall	Why it bites	Do this instead
Bumping retries to 'fix' flakiness	Hides a real race; fails in prod	Reproduce, trace, fix the cause
<code>page.waitForTimeout(3000)</code>	Slow AND still flaky	Web-first assertions auto-wait
<code>trace: 'on'</code> everywhere	Huge artifacts, slow CI	<code>trace: 'on-first-retry'</code>
Tests share global data	Breaks under parallel	Per-test setup / isolated data
html reporter while sharding	Reports can't be merged	<code>blob</code> → <code>merge-reports</code>
Asserting <code>toBeVisible</code> before click	Visible ≠ ready	<code>toBeEnabled()</code> / <code>toBeEditable()</code>

Pattern: every shortcut that 'makes the red go away' without evidence is a future incident.

What you just proved



Reproduced on demand

You converted intermittent failures into reliable ones before fixing them.



Diagnosed from artifacts

Traces, snapshots and network logs named the root cause — no guessing.



Stabilised honestly

Fixes target the cause; retries are scoped to CI noise only.



Scaled the run

Parallel + sharded + merged into a single trustworthy report.

How today's lab is scored (100 pts)

Criterion	What we look for	Pts
Reproduced failures	Both flaky tests fail on demand, trace saved	20
Root-cause accuracy	Cause named & justified from the trace, per test	20
Robust assertions	Web-first / soft assertions + clear messages	15
Retries used correctly	Scoped to CI; not masking a real defect	10
Parallel independence	Full suite green with fullyParallel + workers	15
Sharding + merge	Shards run; single merged report produced	10
Explanation quality	Clear before → after debugging narrative	10
TOTAL		100

Pass = 70+. A test that only passes on retry caps the 'retries used correctly' and 'reproduced' criteria — honesty is scored.

Five things to carry into every suite

- 1 Reproduce before you fix**

An unrepeatable bug can't be proven fixed. Loop it, slow it, isolate it first.
- 2 Let artifacts talk**

Traces, snapshots, network logs and console events name the cause faster than any guess.
- 3 Fix causes, not symptoms**

A retry that hides a race is a deferred production incident, not a fix.
- 4 Web-first assertions**

Gate actions on real state (enabled/editable) — they auto-wait and kill flakiness.
- 5 Independent tests scale**

Isolation is the price of parallelism and sharding — and the biggest reliability win.

Day 5 cheat-sheet

DEBUG & DIAGNOSE

`npx playwright test --ui`

UI mode – time-travel, watch, pick locator

`npx playwright test --debug`

Inspector, headed, step through

`await page.pause()`

Breakpoint in the test code

`npx playwright show-trace trace.zip`

Open a captured trace

`... --project=chromium`

Isolate one browser while reproducing

`DEBUG=pw:api npx playwright test`

Verbose Playwright API logs

STABILISE & SCALE (config)

`retries: process.env.CI ? 2 : 0`

`trace: 'on-first-retry'`

`screenshot: 'only-on-failure'`

`video: 'retain-on-failure'`

`fullyParallel: true`

`workers: process.env.CI ? 4 : undefined`

`reporter: [['list'], ['blob']] // CI`

`--shard=1/3 // CI matrix`

`merge-reports ./all-blob-reports`

DAY 5 COMPLETE

You can now diagnose, stabilise and scale a Playwright suite.

NEXT — DAY 6: WEEKLY ASSESSMENT

Search → Add to Cart → Validate cart

You'll design a complete UI flow with reusability and edge cases — plus an accessibility scan as part of the Gate-1 demo. Bring today's discipline: reproduce, diagnose from artifacts, fix the cause, and keep the suite green AND fast.

Jai Singh